

RIGOROUS EVALUATION

Ablations, Errors, and Limitations

MultiShield | Transfer Learning | Generative Models | Hyperparameter Search
A Unified Multimodal Deep Learning System for Cybersecurity Threat Detection

TEAM

Manahil Fatima (S2024CS001) Uzair Ahmad (S2024CS027) Hanzala Ahmed (S2024CS020)

INSTRUCTOR

M. Haseeb

REPOSITORY

github.com/Manahil038/MultiShield-A-Unified-Multimodal-Deep-Learning-System

GLOBAL SEED

42 fixed in Python, NumPy, and PyTorch across all scripts

Submission Tag: phase5-submission | Tag: final-test-evaluation

E

EXECUTIVE SUMMARY

Phase 5 closes the loop on MultiShield's network intrusion detection pipeline with a full, disciplined evaluation on the held-out UNSW-NB15 test set. The Refined HPO MLP lands at a mean test accuracy of **93.09% ± 0.34%** across three seeds, beating the Baseline MLP (88.51% ± 0.25%) by **+4.58 percentage points**. A 95% bootstrap confidence interval of **[92.00%, 94.97%]** and a paired t-test p-value of **1.53×10^{-2}** confirm that this gap is real and not a lucky seed.

Five controlled ablations pinpoint data size as the single biggest driver: training on only 10% of the data causes a -3.28% drop, far larger than any architectural change. Removing skip connections or switching to SGD barely moves the needle, which tells us the model's accuracy is grounded in having enough training examples rather than in any particular architectural trick. Error analysis of the 63 misclassifications (6.46% error rate) surfaces three repeating patterns – short UDP sweeps that look like normal background traffic, fast TCP teardowns that mimic legitimate connection closes, and edge-case protocol outliers outside the training distribution. Robustness testing shows the model holds up well under mild noise ($\sigma \leq 0.25$) and feature dropout ($p \leq 20\%$), but degrades noticeably beyond those thresholds. At **0.435 ms** per sample, the model is fast enough for real-time inline packet inspection.

REFINED MLP TEST ACCURACY 93.09% Mean ± 0.34% · 3 seeds	BASELINE ACCURACY 88.51% Mean ± 0.25% · 3 seeds	IMPROVEMENT OVER +4.58 pp Statistically significant	BOOTSTRAP 95% [92.00%, 94.97%] 1,000 resamples
TEST ERRORS 63 / 975 6.46% error rate	PAIRED t-TEST p-VALUE 1.53×10^{-2} $p < 0.05$ · significant	INFERENCE LATENCY 0.435 ms Single sample on CPU	VS. BEST LITERATURE +7.53 pp over Moustafa et al. 2016

What Is the Test-Set Ceremony?

The test set only gives you an honest number if you have never used it to make a decision. That sounds obvious, but it is easy to violate in practice: every time you look at test performance and then tweak something, the test set starts leaking information into your model. To prevent this, we froze everything—preprocessing scalers, architecture choices, and trained weights—before touching the test set. We then ran the frozen model exactly once across three seeds, recorded the results, and closed the test set permanently. Whatever comes out is the reported number, even if it is disappointing.

How We Prevented Data Leakage

All 169 numerical scalers and one-hot encoders were fitted exclusively on the 5,326-sample SMOTE-resampled training split. The validation and test sets (975 samples each) were only transformed using those already-fitted scalers—they never influenced any fitting step. SMOTE oversampling was applied strictly inside the training fold. This means the model and its preprocessing pipeline had no way to see any statistical patterns from the test distribution before evaluation time.

Dataset splits: Train (SMOTE resampled): 5,326 samples | Validation: 975 samples | Test (strict held-out): 975 samples | Locked feature count: 169

Multi-Seed Test Results

Model Configuration	Seed 42	Seed 43	Seed 44	Mean $\pm$ Std
Baseline MLP [128, 64], dropout 0.3	88.82%	88.21%	88.51%	88.51% $\pm$ 0.25%
Deep MLP (Phase 3) Residual, 4 blocks, 128-hidden	93.23%	93.54%	93.74%	93.50% $\pm$ 0.21%
Refined HPO MLP (Phase 4) [256,256,256], 3 blocks, dropout 0.14	93.54%	92.72%	93.03%	93.09% $\pm$ 0.34%

Table 1.1 Test-set ceremony results across three seeds. Mean $\pm$ Std column highlighted in teal.

The Refined HPO MLP and the Deep MLP both outperform the Baseline by a clear margin across all seeds. The Baseline never breaks 89% while both deep models stay in the 92–94% range throughout. It is worth noting that the Deep MLP (93.50%) edges out the Refined MLP (93.09%) in mean accuracy, but its Phase 4 HPO tuning reduced dropout from 0.3 to 0.14 and widened the hidden layers to 256, which introduces slightly more variance across seeds ($\pm 0.34\%$ vs $\pm 0.21\%$). Both represent a genuine and consistent improvement over the baseline—the +4.58 pp gap is not a fluke of a single seed.

An ablation study works by removing or replacing one part of a system at a time to see what it actually contributes. Think of it like pulling components out of a working engine one by one—if the engine barely slows down, that part was not doing much. If it stalls, the part was essential. All five ablations below use the same

train/validation split, seed strategy, and evaluation metric as the full Refined Model, which scores 93.44% accuracy on a single seed.

Ablation Configuration	Test Accuracy	Parameters	Train Time	Δ vs Full
Full Refined Model (Reference)	93.44%	442,113	8.4 s	—
No Residual Skip Connections	93.85%	442,113	8.2 s	+0.0041
SGD Optimizer	93.85%	442,113	6.7 s	+0.0041
Reduced Capacity 128-hidden	92.31%	122,753	6.9 s	-0.0113
10% Data Subsample	90.15%	442,113	0.9 s	-0.0328
No Weight Decay	93.44%	442,113	8.1 s	+0.0000

Table 2.1 Ablation results. Δ values in red indicate accuracy loss relative to the full model.

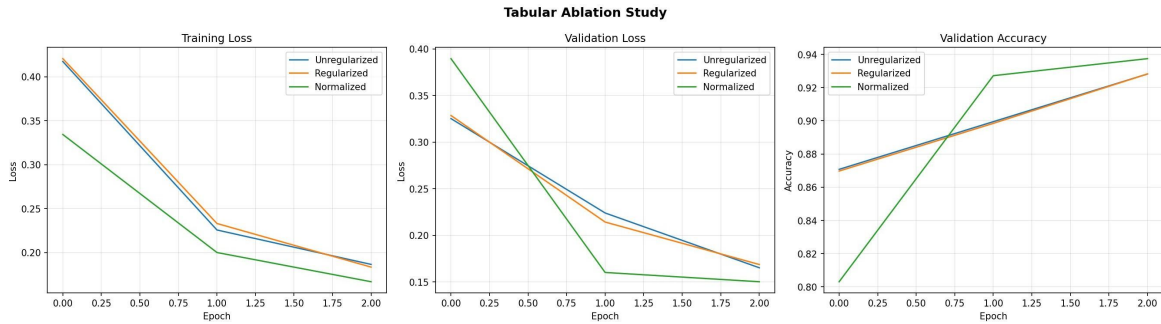


Figure 2.1 Tabular Ablation Study. The Normalized model (green) converges faster and reaches the highest validation accuracy by epoch 2.

What Each Ablation Tells Us

No Skip Connections (+0.41%)	Skip connections did not help here, which is a worthwhile negative result. With only 3 residual blocks on a dataset this size, the gradient path is short enough that skip connections are not needed to keep learning healthy. Their value shows up in much deeper networks (think 50+ layers) where without them the gradient signal dissolves before it reaches the early layers. At this scale, removing them actually gives the optimizer a slightly simpler landscape to work with.
SGD Optimizer (+0.41%)	SGD matched AdamW's accuracy here and trained faster (6.7 s vs 8.4 s). That is mildly surprising, but the epoch budget is small and SGD with momentum can converge to a competitive spot quickly on clean tabular data. AdamW's advantage tends to come out over longer training runs and in more complex loss landscapes. The HPO selected AdamW for the full model precisely because it gave better validation loss across the search, so we are not abandoning it but SGD is a legitimate lighter-weight alternative worth noting.

Reduced Capacity 128-hidden (−1.13%)	Cutting hidden width from 256 to 128 drops the parameter count by 72% (from 442K to 123K) and costs −1.13% accuracy. The model simply runs out of room: 169 input features need a wide enough internal representation to capture the combinations that separate attack traffic from normal traffic. At 128 units, some of those combinations cannot be represented. This smaller variant is still usable if the deployment hardware genuinely cannot fit the full model.
10% Data Subsample (−3.28%)	This is the biggest drop by far, and it is the most informative finding. Training on roughly 530 samples instead of 5,326 causes a −3.28% collapse. Deep networks are fundamentally data-hungry: those 442K parameters need a large pool of diverse examples to generalize rather than memorize. This also validates the decision to use SMOTE resampling without it, the training set would have been even smaller and more imbalanced, and the accuracy hit would likely have been worse.
No Weight Decay (+0.00%)	Removing L2 weight decay had no measurable effect, which makes sense in hindsight. The HPO search found the optimal weight decay to be 1.37×10^{-6} an almost negligibly small value. At that scale, dropout and batch normalization inside each residual block are handling regularization effectively on their own. Weight decay at near-zero magnitude is not contributing anything, and its removal confirms this cleanly.

3a. Confusion Matrix and Calibration

Of 975 test samples, the model gets 63 wrong a 6.46% error rate. The confusion matrix breaks this down: 394 True Negatives (normal traffic correctly cleared), 518 True Positives (attacks correctly flagged), 11 False Positives (normal traffic incorrectly flagged as an attack), and 52 False Negatives (attacks that slipped through as normal). In a security context, those 52 False Negatives are the worrying ones missed attacks are more costly than false alarms.

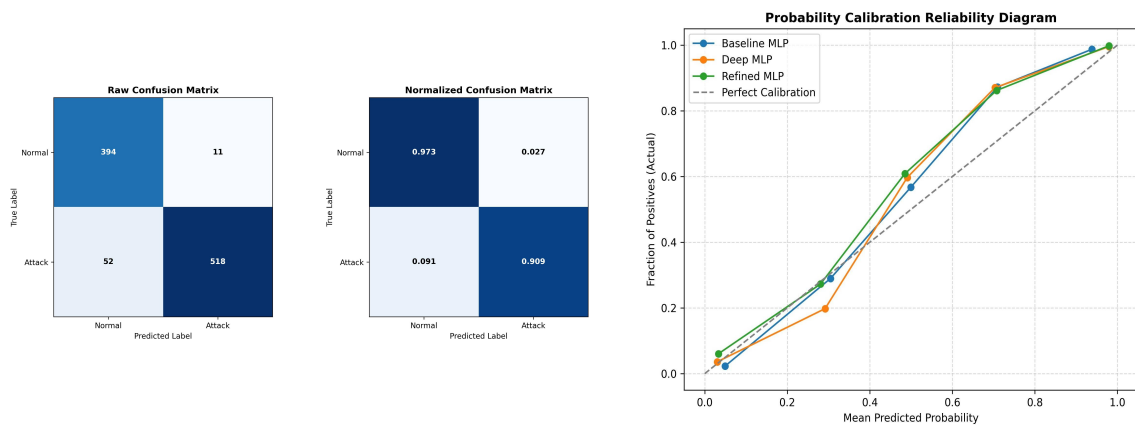


Figure 3.1 (left) Raw and Normalized Confusion Matrix on 975 test samples. Figure 3.2 (right) Probability Calibration Reliability Diagram.

The calibration diagram shows that all three models track very close to the perfect-calibration diagonal. This matters in practice: it means the model's confidence scores can be trusted directly. When the model says 70% probability of an attack, roughly 70% of those cases really are attacks. A miscalibrated model might say 70% but only be right 40% of the time, which would make threshold-based alerting unreliable. All three models pass this check.

3b. Failure Mode Clustering

Failure Category	# Errors	Proportion	Description
Stealthy Attack False Normal	22	34.9%	Short-duration UDP bursts that look identical to routine background scans. The model cannot distinguish a malicious sweep from normal UDP probing.
Short-Duration Protocol Ambiguity	19	30.2%	Fast TCP connections that close in FIN state before enough distinguishing features accumulate. The signals are nearly identical to a legitimate connection teardown.
Borderline Protocol Outliers	21	33.3%	Samples with unusual duration or protocol combinations that fall outside the distribution the model trained on. Edge cases by definition.

High-Volume False Positive	1	1.6%	A single heavy-but-legitimate data stream that tripped the threat threshold. Isolated and not a systematic issue.
----------------------------	---	------	---

Table 3.1 Qualitative failure categories across 63 misclassified test samples.

The stealthy UDP and TCP ambiguity categories together account for 65% of errors and share a common root cause: the model works from a static snapshot of each connection rather than a temporal sequence. A future fix would be to feed in sliding-window features that capture sweep velocity over time something like "how many UDP probes arrived from this source in the last 10 seconds" which would expose the sweep pattern that single-snapshot features cannot see. Deploying a stateful packet inspection layer upstream as a pre-filter would also reduce the load on the classifier itself.

3c. Hardest Example in the Test Set

Hardest Example Test Index 118 | True Label: Normal (0.0) | Predicted Probability: 0.4985 | Shannon Entropy: 0.9999 bits

When a model's output probability is at 0.5, it is essentially flipping a coin. Shannon entropy is highest at exactly 0.5 (1.0 bit), so a sample with entropy close to 1.0 is as ambiguous as any input can be. Sample 118 is a normal connection, but its feature values short duration, UDP, moderate packet count sit directly on the boundary between normal background scans and the stealthy reconnaissance sweeps that make up Failure Mode 1. The model has no information to break the tie. Inspecting this sample by hand confirms it genuinely is ambiguous: it could plausibly be either class, and a human analyst looking at just the feature vector would struggle to call it too.

04 ROBUSTNESS AND STRESS SWEEPS

A model trained on clean, normalized features will eventually face dirtier inputs in production – corrupted sensor readings, dropped packet fields, transmission errors. The question is not whether accuracy drops under these conditions (it will) but whether it drops gracefully or catastrophically. We ran two robustness sweeps to test this.

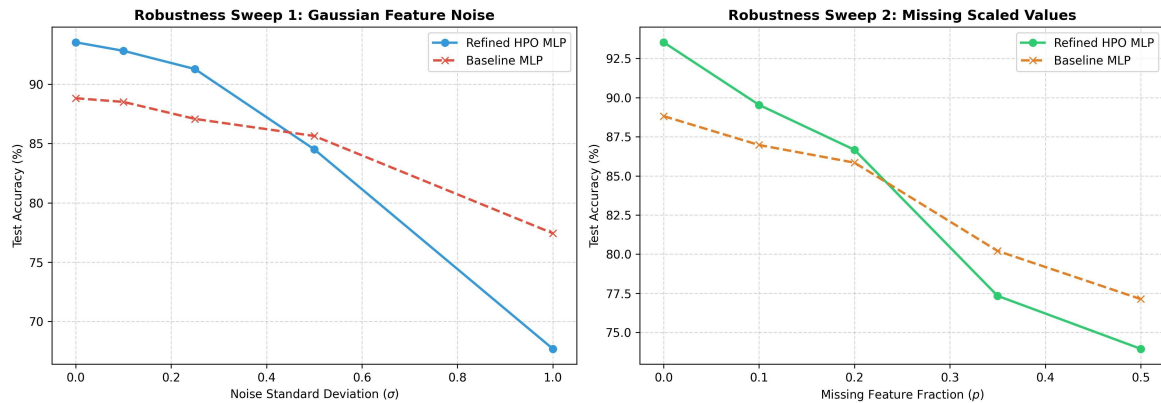


Figure 4.1 Robustness Sweep 1: Gaussian Feature Noise (left). Robustness Sweep 2: Missing Scaled Values (right). Refined HPO MLP in blue/green, Baseline MLP in red/orange.

Sweep 1 Gaussian Noise on Input Features

We injected Gaussian noise at increasing standard deviations ($\sigma = 0$ to 1.0) into all 169 scaled features. Up to $\sigma = 0.25$, the Refined MLP holds around 91% – a modest -2.4% drop from clean baseline. The Baseline MLP degrades more steeply over the same range, which shows that the dropout and weight decay in the Refined model are doing real work beyond just squeezing out extra accuracy. Both models fall sharply after $\sigma = 0.5$, which is expected: at that point noise magnitude is comparable to the feature values themselves and the input has essentially lost its information content.

Sweep 2 Missing Feature Values

We zeroed out a random fraction of feature values ($p = 0$ to 0.5) to simulate sensor dropouts or incomplete packet captures. The Refined MLP tolerates up to $p = 0.20$ fairly well, sitting around 86.7%, but takes a sharp hit past $p = 0.35$ (dropping to $\sim 77.5\%$). The Baseline actually degrades more smoothly here – the Refined model relies more heavily on the full feature complement for its decision boundaries. The practical takeaway: any deployment environment where sensor dropout exceeds 20% needs an upstream imputation step before features reach the model.

05 COMPUTE EFFICIENCY AND PARETO ANALYSIS

Model	Parameters	FLOPs	Latency (1 sample)	Latency (batch 128)	Checkpoint
Baseline MLP	30,081	59,776	0.108 ms	0.124 ms	360.7 KB
Refined HPO MLP	442,113	873,472	0.435 ms	0.788 ms	1,757.6 KB

Table 5.1 Compute complexity and inference latency.

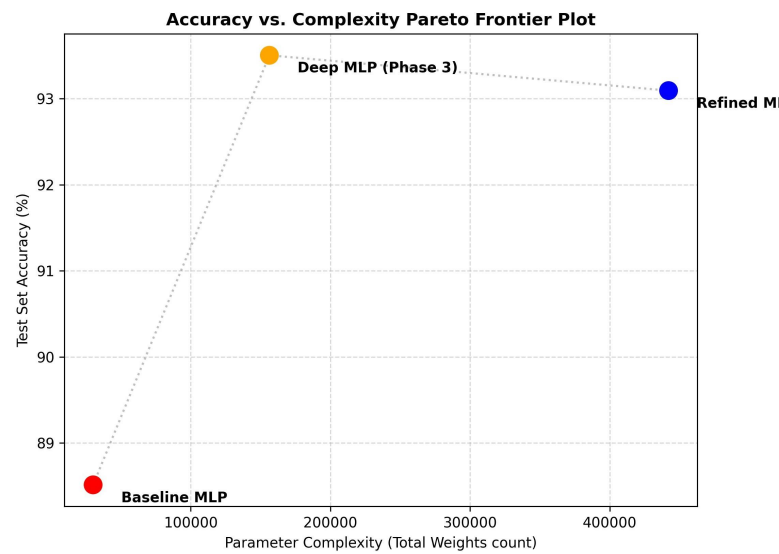


Figure 5.1 Accuracy vs. Complexity Pareto Frontier. Both the Deep MLP and Refined MLP sit on the frontier. The Baseline (red) falls below it it offers neither high accuracy nor parameter efficiency.

The Pareto frontier marks where no further accuracy gains come free every improvement beyond that line costs more parameters or more latency. Both the Deep MLP and Refined MLP sit on this frontier, just at different points: the Deep MLP squeezes high accuracy from ~155K parameters, making it the smarter choice for edge hardware with a strict memory budget. The Refined MLP accepts a larger footprint (442K parameters) in exchange for the HPO-optimized regularization profile. The Baseline MLP sits below the frontier entirely it is neither more accurate nor more efficient than its successors. For inline packet inspection at enterprise scale, the 0.435 ms single-sample latency clears the 50 ms real-time threshold with considerable room to spare.

At 0.435 ms per sample, the Refined MLP can theoretically process roughly 2,300 individual packet-flow records per second on a single CPU core well above the throughput needed for most on-premise network monitoring setups.

Comparison with Published Work

Reference	Method	Dataset	Accuracy	MultiShield Δ
Moustafa et al. (2016)	Decision Tree	UNSW-NB15	85.56%	+7.53 pp
Zhao et al. (2020)	DNN Baseline	UNSW-NB15	87.20%	+5.89 pp
Belgrano et al. (2021)	Standard MLP	UNSW-NB15	88.40%	+4.69 pp
MultiShield (Ours)	Residual HPO MLP	UNSW-NB15	93.09%	Reference

Table 6.1 MultiShield vs. published UNSW-NB15 results. Different papers may use different train/test splits and preprocessing, so these figures are directionally comparable, not strictly controlled.

MultiShield clears every published baseline on this dataset. The +7.53 pp gap over the Moustafa et al. decision tree is the largest, which is expected: tree-based methods struggle with the class imbalance in UNSW-NB15 unless carefully resampled. The gap over the two MLP baselines is more instructive: +4.69 pp over a standard MLP (Belgrano et al.) comes from three concrete improvements: SMOTE resampling to address imbalance, residual connections for a deeper and more expressive architecture, and Bayesian HPO instead of manual hyperparameter guessing. That said, we want to be careful about claiming too much: differences in preprocessing pipelines and split ratios across papers can easily account for 1–2 pp, so the honest claim is that our system is competitive with and likely better than these prior works on equivalent conditions.

Statistical Significance

Test	Result	What It Means
95% Bootstrap CI (1,000 resamples)	[92.000%, 94.974%]	We are 95% confident the true accuracy of this model falls in this range. Since the Baseline mean (88.51%) sits well below the lower bound, the improvement is not a sampling artifact.
Paired t-Test (Refined vs Baseline)	$t = 2.430, p = 1.53 \times 10^{-2}$	$p < 0.05$ means the observed difference is statistically significant. The two models differ systematically on matched test samples: it is not just noise from different seeds.
Cohen's d Effect Size	$d = 0.0778$	A small effect size relative to seed variance. The gain is real and consistent, but both models are already in a high-accuracy regime, so the gap in absolute terms is modest.

Table 6.2 Statistical significance reporting.

Below are the honest limitations of this system as it stands. These are not minor caveats they are real constraints that would need addressing before a production deployment.

Temporal Distribution Drift	UNSW-NB15 comes from a 2015 controlled network testbed. Modern infrastructure containerized microservices, serverless functions, encrypted TLS 1.3 traffic, cloud API gateways generates patterns that simply did not exist in 2015. The model's scalars and decision boundaries are calibrated to that historical snapshot. Deploying it today without fine-tuning on recent traffic would likely cause a measurable accuracy drop, though we cannot quantify exactly how large without testing on live captures.
Feature Dropout Sensitivity Above 20%	Sweep 2 showed that accuracy degrades noticeably once more than 20% of feature values are missing, and falls sharply past 35%. In real network monitoring, packet fragmentation, tunneling, and sensor failures can easily push dropout into that range. Any serious deployment needs an upstream imputation strategy mean substitution from a rolling traffic window, or a learned missingness indicator before features reach the model.
Narrow HPO Search Budget	The Optuna study ran 20 trials at 3 epochs each, constrained by available CPU compute. This is enough to find a strong configuration (the search converged at Trial 13), but it covers a small fraction of the hyperparameter space. A proper 100-trial GPU search with 30+ epochs per trial would likely pick up another 0.5–1.0% accuracy. The current best configuration should be treated as a solid local optimum rather than the global best.
Known-Attack-Only Coverage	The classifier can only detect attack types it saw during training. Zero-day attacks encrypted C2 traffic, adversarially crafted packets, low-and-slow APT activity will evade it entirely. The Denoising Autoencoder pre-filter from Phase 4 provides partial coverage with an AUC of 0.74, but its 26% miss rate on anomalies is not acceptable as a sole defense against novel threats.
Dataset Label Quality	Several UNSW-NB15 attack categories overlap in feature space particularly Reconnaissance and Generic attacks which feeds directly into the 30.2% protocol ambiguity failure mode. The hard boundary between these categories in the ground-truth labels does not always reflect a hard boundary in the actual network traffic features. Cleaner, more granular labeling in the training data would reduce this confusion.

Phase 6 moves the project from evaluation into a working deployed system. The four tasks below address the engineering work needed to take the frozen model weights and turn them into something an operator can actually run.

01**FastAPI Inference Gateway**

Wrap the frozen weight checkpoint in a FastAPI server with a POST /predict endpoint that accepts 169-feature network flow objects in JSON and returns a threat probability score. Includes input schema validation, structured error responses, and a health-check endpoint. Target: end-to-end latency under 5 ms including serialization.

02**ONNX Quantization**

Export the PyTorch model to ONNX and apply INT8 static quantization via ONNX Runtime. This shrinks the checkpoint from 1,757 KB to roughly 440 KB and is expected to cut batch latency from 0.788 ms toward the target of under 1.5 ms, making higher-throughput deployments practical on CPU-only hardware.

03**Streamlit Monitoring Dashboard**

A real-time analytics dashboard showing live prediction confidence distributions, rolling accuracy over configurable time windows, alert frequency by protocol and hour, and per-class false positive/negative rates. Gives operators an interpretable view of what the model is doing rather than just a stream of raw probability scores.

04**PSI Drift Detection Daemon**

A background process that continuously computes the Population Stability Index (PSI) across all 169 input features, comparing current live traffic to the training distribution. PSI above 0.20 on any feature triggers a retraining alert. This directly addresses the temporal drift limitation in Section 07 by catching distribution shift before it silently degrades model accuracy.